Department of Software Engineering, NED University of Engineering & Technology
SE-303 Operating Systems — Lab Session 13 (Complex Engineering Activity)

Design and Evaluation of a Multi-Level Feedback Queue Scheduler for xv6-RISC-V: Impact on CPU-Bound and I/O-Bound Workloads

Muhammad Ashhadullah Zaheer (SE-24087) · Misbah Sarfraz (SE-24096)
Batch 2024–2028

Abstract

This report describes the design, implementation and evaluation of a three-level Multi-Level Feedback Queue (MLFQ) scheduler for the xv6-RISC-V teaching kernel, together with a new system call, `getprocinfo()`, that exposes per-process CPU accounting. The scheduler was compared with the stock round-robin (RR) policy using a CPU-bound and an I/O-bound workload under deterministic, instruction-count-based virtual time. When one I/O-bound process competes with three CPU-bound processes, MLFQ reduces its completion time from 374.7 to 39.0 ticks (a 9.6× improvement), while the CPU-bound processes take 8.9% longer. Mean completion time across all processes falls by 17%, and the time needed to finish all work (makespan) is essentially unchanged. An ablation study shows that the benefit depends entirely on preempting the running process when a higher-priority process wakes up; without that mechanism, MLFQ performs no better than RR.

1. Introduction

The unmodified xv6 kernel uses a round-robin scheduler: every runnable process receives the CPU in turn, irrespective of how it uses it. Such a policy is fair in CPU share but penalises processes that block frequently, since each time they become runnable they must wait behind every competing process. MLFQ, first introduced in the Compatible Time-Sharing System [3] and described in standard texts [2, 4], addresses this by inferring a process's behaviour from its history and favouring processes that relinquish the CPU early.

The objectives of this activity were to (i) replace the xv6 scheduling policy [1] with an MLFQ, (ii) add a system call that reports per-process scheduling statistics, (iii) build representative CPU-bound and I/O-bound test applications, and (iv) quantify how the change affects each class of workload and what this implies for application design.

2. Experimental Setup

Table 1. Experimental environment

Item	Configuration
Kernel	xv6-riscv (github.com/mit-pdos/xv6-riscv), commit fa2ddcb
Emulator	QEMU 8.2.2 [5], machine *virt*, 128 MB RAM, virtio block device
Toolchain	riscv64-linux-gnu-gcc 13.3.0 on Ubuntu 24.04
CPUs (benchmarks)	One hart (`-smp 1`), so both policies arbitrate a single CPU
Clock source	QEMU `-icount shift=2`: virtual time derived from executed instructions rather than the host clock
Timer interrupt	1,000,000 cycles = 100 ms of virtual time (xv6 default)
Repetitions	Three runs per scenario and kernel variant

Three kernels were built from a single source tree: **RR**, the original round-robin loop (`make SCHED=RR`); **MLFQ**, the final design; and **MLFQ-nopreempt**, MLFQ with wakeup preemption disabled (`make NOPREEMPT=1`), used solely for ablation. The accounting code is identical in all three variants, so any

difference in the measurements is attributable to the scheduling decision alone.

Clock calibration. An initial measurement campaign used `-icount shift=2,sleep=off`. In this mode QEMU advances virtual time instantly whenever the CPU is idle, which inflates the apparent waiting time of processes blocked on the disk: a solo I/O run reported more than 5,500 ticks instead of approximately 34, with large run-to-run variation. These results were discarded. With QEMU's default idle behaviour, repeated runs agree within one or two ticks, which is the precondition for a meaningful comparison.

Baseline. The unmodified xv6 source was built independently and booted to the shell (`logs/baseline_unmodified_boot.txt`). The RR kernel is identical to it apart from the added accounting, and its solo measurements (Table 5) serve as the baseline.

3. Design and Implementation

3.1 Scheduling policy

The scheduler maintains three priority levels, with level 0 the highest. The time quantum doubles at each level — 1, 2 and 4 ticks for levels 0, 1 and 2 — so that short, interactive work is dispatched frequently while long computations incur fewer context switches. The policy is defined by four rules:

1 A newly created process enters level 0. Because this is set in `allocproc()`, a child always starts at the top regardless of its parent's level.

2 The scheduler always selects a runnable process from the highest non-empty level. Within a level, the least recently dispatched process runs first, yielding round-robin among equals.

3 A process preempted after consuming its full quantum is demoted one level; level 2 is the lowest.

4 A process that yields the CPU before its quantum expires — by sleeping on the disk, a pipe or `pause()`, or by exiting — retains its level and its quantum usage is reset.

3.2 Extensions beyond the specification

Periodic priority boost. Every 100 ticks, all processes are returned to level 0. Without this rule, a process demoted during a long computation could be starved indefinitely by a continuous stream of interactive processes.

Preemption on wakeup. When `wakeup()` makes runnable a process whose level is higher than that of the process currently running on a CPU, that CPU is flagged to reschedule on its next trap exit. Stock xv6 invokes the scheduler only on timer interrupts, so without this mechanism a newly woken high-priority process could wait up to 100 ms behind a CPU-bound process. Sections 6 and 7 quantify its effect.

3.3 CPU-time accounting

The first implementation charged a full tick to whichever process was running when the timer fired, demoting it once its tick count reached the quantum. The functional tests exposed a flaw in this approach: an I/O-bound process that ran for only microseconds per wakeup was occasionally demoted simply because a timer interrupt coincided with one of those short bursts. The final design instead reads the RISC-V `time` CSR immediately before and after each dispatch in `scheduler()` and charges the measured interval to the process. If the process returns still runnable, it was preempted and the interval counts toward its quantum; if it returns sleeping or exiting, the quantum counter is reset. The same measurement feeds `getprocinfo()`, so reported CPU time is exact rather than sampled.

3.4 The getprocinfo system call

`int getprocinfo(int pid, struct procinfo *info)` populates a structure containing the process ID, state, scheduler mode, current level, CPU time in ticks and milliseconds, dispatch count, and CPU milliseconds spent at each level. A `pid` of zero or less refers to the caller. Since an xv6 system call returns a single integer, the signature specified in the lab, `getprocinfo(int pid)`, was extended with an output pointer so that multiple metrics can be returned. The kernel snapshots the data while holding the process lock and performs `copyout()` only after releasing it. The call returns −1 for a non-existent process or an invalid user pointer.

3.5 Concurrency and kernel integration

All new per-process fields reside in `struct proc` and are protected by `p->lock`, consistent with the existing fields of that structure. The selection routine scans the process table once, holding each lock only briefly; it then re-acquires the lock of the chosen process and confirms that it is still runnable before switching to it, rescanning if another CPU has claimed it in the meantime. The only lock-free access is the priority comparison in `wakeup()`, which sets a per-CPU hint flag; a stale read can at worst cause one superfluous or one delayed reschedule, never incorrect behaviour. The system call follows the established xv6 pattern: number in `syscall.h`, dispatch entry in `syscall.c`, handler in `sysproc.c`, stub in `usys.pl` and prototype in `user.h`. Table 2 summarises the changes.

Table 2. Summary of source changes

File(s)	Change
kernel/proc.c, proc.h	MLFQ selection, run-time accounting, priority boost, wakeup preemption hint, kgetprocinfo(), new per-process fields
kernel/trap.c	Yield on trap exit when preemption is requested; boost invoked from the clock interrupt
kernel/sysproc.c, syscall.c, syscall.h, user/usys.pl, user/user.h	getprocinfo system call
kernel/procinfo.h, param.h, defs.h	Shared result structure; tunables (NQUEUE, MLFQ_BOOST_TICKS, TICK_CYCLES); prototypes
Makefile	SCHED=RR and NOPREEMPT=1 build options; new user programs
user/cpubound.c, iobound.c, workloads.h	Test applications and shared workload code
user/bench.c, mlfqtest.c, mlfqdemo.c	Benchmark driver, functional tests, scheduling demonstration

4. Test Applications

cpubound counts the primes below 4,000,000 by trial division. It issues no system calls during computation, so it never blocks and always exhausts its quantum. On a single hart it requires approximately 11.4 s of CPU time (115 ticks).

iobound performs 100 iterations of the sequence: create a file, write four 64-byte blocks, close it, and delete it. Each operation passes through the xv6 log layer and the virtio disk driver and puts the process to sleep. Consequently the program is dispatched roughly 4,423 times per run while holding the CPU only briefly each time. Run alone, it completes in 33 ticks.

bench `N M` forks *N* instances of cpubound and *M* of iobound and starts them simultaneously. Each child records its start and finish times along with its `getprocinfo()` counters and reports them to the parent through a pipe, ensuring that output from concurrent processes is never interleaved. During development, an early version of the CPU workload completed in 0 ms because the compiler eliminated a loop whose result was unused; the result is now written to a `volatile` variable to prevent this optimisation.

5. Verification

5.1 Functional tests

The `mlfqtest` program performs the 17 checks listed in Table 3. All checks pass on the MLFQ kernel in both single-hart and three-hart configurations (`logs/functional_test_mlfq.txt`, `logs/functional_test_mlfq_3cpu.txt`).

Table 3. Functional test results (identical for one and three harts)

#	Check	Result
1	getprocinfo(self) succeeds	PASS
2	Reported pid equals getpid()	PASS
3	Kernel reports MLFQ mode	PASS
4	New process starts at priority 0	PASS
5	nsched ≥ 1 for a running process	PASS
6	getprocinfo(explicit own pid) matches self query	PASS
7	getprocinfo(invalid pid 99999) returns −1	PASS
8	getprocinfo(invalid pointer) returns −1	PASS
9	cpu_ticks increases while spinning	PASS
10	CPU-intensive process demoted below level 0	PASS
11	CPU-intensive process reaches lowest level	PASS
12	Sum of level_ms equals cpu_ms	PASS
13	Time at lowest level is recorded	PASS
14	Child of a demoted parent starts at priority 0	PASS
15	I/O-bound process remains at priority 0	PASS
16	Sleeping process remains at priority 0	PASS
17	Periodic boost restores demoted process to level 0	PASS

5.2 Regression testing

To confirm that the changes do not destabilise the kernel, the standard xv6 test suite (`usertests -q`) was run on the MLFQ kernel with three harts. It completed with “ALL TESTS PASSED” and no kernel panic (`logs/usertests_mlfq.txt`).

5.3 Behavioural demonstration

The `mlfqdemo` program launches two CPU-bound processes (A and B) and one I/O-bound process and samples their state every two ticks. Table 4 contrasts the two kernels; each cell shows queue level / CPU ticks / dispatch count (`logs/demo_rr.txt`, `logs/demo_mlfq.txt`).

Table 4. Scheduler behaviour over the first 18 ticks (level / CPU ticks / dispatches)

Tick	CPU-A (RR)	CPU-B (RR)	I/O (RR)	CPU-A (MLFQ)	CPU-B (MLFQ)	I/O (MLFQ)
2	L0 / 0 / 1	L0 / 1 / 1	L0 / 0 / 1	L0 / 0 / 1	L0 / 0 / 1	L0 / 0 / 1
4	L0 / 1 / 2	L0 / 2 / 2	L0 / 0 / 2	L1 / 1 / 2	L1 / 1 / 2	L0 / 0 / 2
6	L0 / 2 / 3	L0 / 3 / 3	L0 / 0 / 3	L1 / 1 / 124	L1 / 2 / 123	L0 / 1 / 246
8	L0 / 3 / 4	L0 / 4 / 4	L0 / 0 / 4	L1 / 1 / 243	L1 / 2 / 243	L0 / 2 / 487
10	L0 / 4 / 5	L0 / 5 / 5	L0 / 0 / 5	L1 / 1 / 357	L1 / 2 / 357	L0 / 3 / 717
12	L0 / 5 / 6	L0 / 6 / 6	L0 / 0 / 6	L1 / 2 / 474	L1 / 2 / 473	L0 / 5 / 952
14	L0 / 5 / 7	L0 / 7 / 7	L0 / 0 / 7	L1 / 2 / 589	L1 / 3 / 588	L0 / 6 / 1184
16	L0 / 6 / 8	L0 / 8 / 8	L0 / 0 / 8	L1 / 2 / 705	L1 / 3 / 705	L0 / 7 / 1419
18	L0 / 7 / 9	L0 / 9 / 9	L0 / 0 / 9	L1 / 2 / 818	L1 / 3 / 818	L0 / 8 / 1646

Under RR, every process remains at level 0 and the I/O-bound process has been dispatched only nine times by tick 18, because after each disk operation it must queue behind both CPU-bound processes. Under MLFQ, the CPU-bound processes are demoted within the first few ticks while the I/O-bound process stays at level 0 and has been dispatched 1,646 times in the same interval — roughly 180 times as often.

6. Results

Completion time is reported in timer ticks (1 tick = 100 ms of virtual time), measured by each process from its own start to its own exit. Values are the mean of three runs; the range is shown in parentheses where runs differed. CPU time is the process's own consumption as reported by `getprocinfo()`, and dispatches is the number of times the scheduler switched to it.

Table 5. Each program running alone (baseline)

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, RR	115	11,437	116	0
CPU-bound, MLFQ	115	11,436	116	2
I/O-bound, RR	33.3 (33–34)	2,380	4,423	0
I/O-bound, MLFQ	35	2,381	4,426	0

Table 6. One CPU-bound and one I/O-bound process started together

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, RR	116	11,470	117	0
I/O-bound, RR	147.3 (147–148)	2,380	4,426	0
CPU-bound, MLFQ	145.3 (145–146)	11,638	4,511	2
I/O-bound, MLFQ	37.3 (37–38)	2,381	4,426	0
CPU-bound, MLFQ-nopreempt	116	11,464	117	2
I/O-bound, MLFQ-nopreempt	149	2,380	4,424	0

Table 7. Three CPU-bound and one I/O-bound process started together

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, RR	343.3 (342–345)	11,433	116	0
I/O-bound, RR	374.7 (374–375)	2,380	4,423	0
CPU-bound, MLFQ	373.7 (372–375)	11,506	1,581	2
I/O-bound, MLFQ	39	2,381	4,426	0
CPU-bound, MLFQ-nopreempt	345.7 (345–347)	11,457	117	2
I/O-bound, MLFQ-nopreempt	376.7 (376–377)	2,380	4,425	0

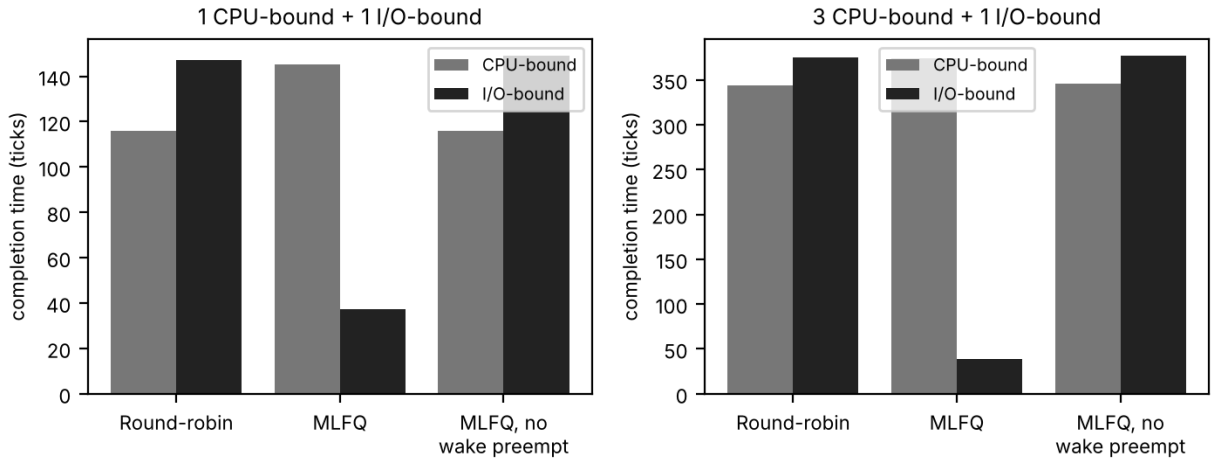


Figure 1. Mean completion time of each process class under each kernel variant (three runs).

Table 8. Summary of the mixed-workload experiments

Metric	1 CPU + 1 I/O RR	1 CPU + 1 I/O MLFQ	3 CPU + 1 I/O RR	3 CPU + 1 I/O MLFQ
I/O-bound completion (ticks)	147.3	37.3	374.7	39.0
I/O-bound slowdown vs. solo RR run (33.3 ticks)	4.4×	1.1×	11.2×	1.2×
CPU-bound completion (ticks, mean)	116.0	145.3	343.3	373.7
Mean completion, all processes (ticks)	131.7	91.3	351.2	290.0
Makespan: time until last process exits (ticks)	147.3	145.3	374.7	375.0

7. Discussion

7.1 I/O-bound workload: large benefit

Running alone, the I/O-bound program completes in 33 ticks. Alongside three CPU-bound processes under RR it requires 374.7 ticks — an 11.2× slowdown — and finishes only after the CPU-bound processes have exited. Each of its roughly 4,423 dispatches follows a wakeup, and under RR every wakeup places it at the back of the run queue, behind processes that each hold the CPU for a full tick. Under MLFQ the same program completes in 39 ticks, only 1.2× its solo time. Because it never exhausts a quantum, it is never demoted (its final level is 0 in every run) and is always selected ahead of the lower-priority CPU-bound processes.

7.2 CPU-bound workload: modest cost

Each CPU-bound process consumes its entire quantum on every dispatch and is therefore demoted to level 2 in every MLFQ run, executing only when no higher-priority work is runnable. In the three-process scenario, their mean completion time rises from 343.3 to 373.7 ticks (+8.9%). This cost has two components. The first is the CPU time yielded to the I/O-bound process, about 2.4 s (24 ticks). The second is additional context-switch overhead: each CPU-bound process is dispatched 1,581 times instead of 116, because every I/O completion preempts it. The makespan is effectively identical under both policies (374.7 versus 375.0 ticks), which confirms that MLFQ does not waste CPU capacity; it only changes the order in which work is completed.

7.3 Aggregate effect

Completing short jobs first lowers the mean completion time from 351.2 to 290.0 ticks (–17%) in the three-process scenario and from 131.7 to 91.3 ticks (–31%) in the two-process scenario. This reproduces the benefit of shortest-job-first scheduling without requiring advance knowledge of job lengths: the scheduler infers from observed behaviour that a process which repeatedly blocks has short CPU bursts.

7.4 Wakeup preemption is essential

With wakeup preemption disabled, the I/O-bound process completes in 376.7 ticks in the three-process scenario and 149 ticks in the two-process scenario, within two ticks of RR (374.7 and 147.3). Priority levels influence behaviour only at the instants when the scheduler runs, and stock xv6 runs it once per tick. A priority-based policy layered on a tick-driven kernel therefore requires a mechanism that invokes the scheduler as soon as a higher-priority process becomes runnable. The policy alone is not sufficient; the kernel mechanism that enforces it determines whether applications observe any benefit.

8. Implications for Application Design

- **Blocking early is rewarded.** Programs that spend most of their time waiting and compute in short bursts retain high priority and remain responsive under load, whereas long computations should expect to run at low priority whenever interactive work is present. This favours event-driven designs that block on input and perform small units of work, and it penalises batch jobs that attempt to appear responsive by polling in a busy loop.
- **Long computations are preempted frequently.** The CPU-bound processes were dispatched about 1,581 times instead of 116. Workloads that keep their state in memory and resume cheaply are barely affected, but those that rely on long, uninterrupted, cache-friendly execution lose more.

- **The policy can be gamed.** A CPU-bound program that sleeps or issues a trivial I/O request just before its quantum expires is classified as interactive and stays at level 0. The periodic boost prevents starvation but not this abuse. Production schedulers mitigate it by charging cumulative time at a level against a fixed allotment rather than resetting the counter on every yield [2]; this rule was not implemented here.
- **Classification lags behaviour changes.** A process that computes for a long period and then becomes interactive remains at a low level until the next boost, so its first responses are slow; conversely, a process that begins with I/O and then computes is demoted quickly. Separating I/O handling and computation into distinct processes allows each to be scheduled according to its actual behaviour.
- **Parameters are part of the design.** The quanta (1, 2 and 4 ticks) and the boost interval (100 ticks) were chosen analytically and were not tuned. A different tick length or I/O latency could change the results, and the benchmark framework makes these parameters straightforward to vary.

9. Limitations

All results were obtained on an emulated machine with a fast virtual disk, a single hart and instruction-count-based virtual time. Absolute tick counts on physical hardware or a different QEMU version will differ; the relative comparisons are the meaningful outcome. Only two workload types were studied. The scheduler scans all 64 process-table slots on every decision, as the original loop does, and is therefore not intended for large process tables. The benchmarks were run on one hart, and multi-hart behaviour was validated only through the functional tests and a single run of `usertests`.

10. Reproducibility

The measurements were taken with `-icount shift=2` appended to `QEMUOPTS` in `xv6-riscv/Makefile`; the default Makefile does not include this option. The raw per-run data are in `results/results.csv` and the console logs in `logs/`.

```
cd xv6-riscv
make qemu CPUS=1                              # MLFQ kernel (default)
make clean && make qemu CPUS=1 SCHED=RR       # round-robin baseline
make clean && make qemu CPUS=1 NOPREEMPT=1    # MLFQ without wakeup preemption

# inside the xv6 shell
bench 1 0      # CPU-bound alone          bench 0 1   # I/O-bound alone
bench 1 1      # 1 CPU + 1 I/O            bench 3 1   # 3 CPU + 1 I/O
mlfqtest       # functional tests         mlfqdemo    # behaviour trace
usertests -q   # xv6 regression suite
```

References

[1] R. Cox, M. F. Kaashoek and R. Morris, *xv6: a simple, Unix-like teaching operating system* (RISC-V edition). MIT PDOS. Available: https://github.com/mit-pdos/xv6-riscv

[2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, “Scheduling: The Multi-Level Feedback Queue,” ch. 8 in *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.

[3] F. J. Corbató, M. Merwin-Daggett and R. C. Daley, “An experimental time-sharing system,” in *Proc. AFIPS Spring Joint Computer Conference*, 1962, pp. 335–344.

[4] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[5] QEMU Project, “QEMU System Emulation User’s Guide.” Available: https://www.qemu.org/docs/master/system/

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
(SE-303) Operating Systems

Student: Muhammad Ashhadullah Zaheer (SE-24087)

Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
1. Scheduler Logic & Correctness	The MLFQ scheduler is implemented correctly and efficiently, adhering to all specified logic (priority levels, promotion/demotion rules).	The scheduler is functional but contains minor logical flaws or inefficiencies. It may not fully handle all edge cases correctly.	The scheduler fails to compile, is non-functional, or does not implement the core requirements of an MLFQ.	CPA1 CPA2 CPA3	
2. Kernel Integration & Stability	The new scheduler and system call are cleanly integrated into the kernel without causing instability, memory leaks, or race conditions.	The new code is functional but may cause minor, intermittent issues or does not follow xv6's existing design patterns.	The implementation causes frequent kernel panics, data corruption, or fails to integrate with the existing kernel code.	CPA1 CPA2 CPA3	
3. Test Application Design	Both the CPU-bound and I/O-bound applications are well-designed, effectively simulating their respective workloads to clearly test the scheduler.	The applications are functional but are not distinct enough in their behavior, making the performance difference less clear.	The applications fail to run, do not correctly simulate the intended workloads, or were not developed.	CPA1 CPA2 CPA3	
4. Functional Demonstration	The functionality of the new scheduler is clearly and correctly demonstrated, showing how it prioritizes processes differently than the original scheduler.	A demonstration is provided, but it is unclear or fails to convincingly show the impact of the new scheduling algorithm.	No functional demonstration is provided, or the demonstration shows the implementation does not work.	CPA1 CPA3	
5. Code Quality & Readability	The code written (both kernel and user-level) is clean, well-commented, easy to understand, and follows standard C programming practices.	The code is functional but is poorly formatted, lacks sufficient comments, or is difficult to follow.	The code is unreadable, uncommented, and does not follow basic programming standards.	CPA3	
Weighted CLO Score					
Remarks and Signature					

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
(SE-303) Operating Systems

Student: Misbah Sarfraz (SE-24096)

Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
1. Scheduler Logic & Correctness	The MLFQ scheduler is implemented correctly and efficiently, adhering to all specified logic (priority levels, promotion/demotion rules).	The scheduler is functional but contains minor logical flaws or inefficiencies. It may not fully handle all edge cases correctly.	The scheduler fails to compile, is non-functional, or does not implement the core requirements of an MLFQ.	CPA1 CPA2 CPA3	
2. Kernel Integration & Stability	The new scheduler and system call are cleanly integrated into the kernel without causing instability, memory leaks, or race conditions.	The new code is functional but may cause minor, intermittent issues or does not follow xv6's existing design patterns.	The implementation causes frequent kernel panics, data corruption, or fails to integrate with the existing kernel code.	CPA1 CPA2 CPA3	
3. Test Application Design	Both the CPU-bound and I/O-bound applications are well-designed, effectively simulating their respective workloads to clearly test the scheduler.	The applications are functional but are not distinct enough in their behavior, making the performance difference less clear.	The applications fail to run, do not correctly simulate the intended workloads, or were not developed.	CPA1 CPA2 CPA3	
4. Functional Demonstration	The functionality of the new scheduler is clearly and correctly demonstrated, showing how it prioritizes processes differently than the original scheduler.	A demonstration is provided, but it is unclear or fails to convincingly show the impact of the new scheduling algorithm.	No functional demonstration is provided, or the demonstration shows the implementation does not work.	CPA1 CPA3	
5. Code Quality & Readability	The code written (both kernel and user-level) is clean, well-commented, easy to understand, and follows standard C programming practices.	The code is functional but is poorly formatted, lacks sufficient comments, or is difficult to follow.	The code is unreadable, uncommented, and does not follow basic programming standards.	CPA3	
Weighted CLO Score					
Remarks and Signature					